

Day 1 — Local dev, debugging, repo/scripts

1) “Local app debugging and URL not working by missing api version and sig”

This usually happens when calling **Azure Storage REST** with a **SAS URL**:

- **sig** = SAS signature (part of the SAS token)
- **api-version** / **x-ms-version** = Storage API version header/query expected by some examples

Fixes

- Prefer **Azure SDK** (Blob SDK) instead of hand-building URLs.
- If you must use REST/SAS:
 - Ensure the full SAS token is present: `?sv=...&ss=...&srt=...&sp=...&se=...&st=...&spr=...&sig=...`
 - Include required headers like `x-ms-version` and `x-ms-date` (depending on the REST call).
- If developing locally, use **Azurite** for Storage emulation.

2) Breakpoints for Logic App

Logic Apps don't support “breakpoints” like code, but you can debug effectively:

- Use **Run history** → inspect **Inputs/Outputs** per action.
- Add **Compose** actions to print intermediate values.
- Use **tracked properties** (Tracking section) to surface key values in logs.

For **Logic App Standard local debugging**:

- Open the project in VS Code (Logic Apps extension).
- Run the local runtime (Functions host) and trigger the workflow locally.
- Inspect run history locally in the extension's local run output.

3) Breakpoints for Function App (Python)

- Use VS Code with the Python extension.
- Start functions host: `func host start`
- Attach debugger (debugpy). If needed, add an attach config like:

```
{
  "name": "Attach to Python Functions",
  "type": "debugpy",
  "request": "attach",
  "connect": { "host": "localhost", "port": 9091 },
  "preLaunchTask": "func: host start"
}
```

Day 2 — Logic App + PostgreSQL (private / VNet), dynamic SQL, filtering

1) Postgres DB is private — access via VNet

Logic App Standard (single-tenant) is the usual choice for private networking:

- Enable **VNet Integration** so the workflow can reach private endpoints/private IPs.
- Ensure **Private DNS** is configured so the Postgres hostname resolves to the private IP.

Azure Functions:

- Use **Premium** (or App Service plan) + VNet Integration for private access.

2) “Execute a SQL query” with dynamic content binding

Use **parameterized** queries:

- Avoid string concatenation (SQL injection risk).
- Bind values via connector parameters.

Example idea:

- Query: `SELECT * FROM products WHERE category = $1;`
- Parameter `$1` : from HTTP trigger (query/body)

3) “Filter/select query condition (not whole table data)”

Best practice:

- Filter at the database using WHERE + indexes
- Add paging/limits (e.g., LIMIT/OFFSET)
- Only return required fields (projection)

4) “How to check return type of each action? e.g., Get Rows → List of Items?”

In **Run history**, open the action and inspect **Outputs**.

Common helpers:

- `@outputs( 'Get_rows' )` → full output object
- `@body( 'Get_rows' )` → body only

Tip: add a **Compose** after “Get rows” to see the shape quickly.

5) “In real world we don’t return DB data as-is”

Pattern:

- Logic App receives request and calls an **Azure Function**
- Function queries DB, **transforms into API DTO**, returns clean JSON
- Logic App returns that JSON to the caller

Day 2 — Key Vault and app settings

1) “Where did you define env variables for local development?”

Azure Functions

- Local: `local.settings.json`
- Azure: Function App → **Configuration** → Application settings
- Read in Python: `os.environ["NAME"]`

Logic App Standard

- Local: `local.settings.json` in the Standard project
- Azure: Logic App Standard → **Configuration** → Application settings
- Use in workflow: `@appsetting('NAME')`

2) “How to get the secret link (Key Vault)?”

In Key Vault:

- Open the **Secret**
- Open a **version**
- Copy the **Secret Identifier (URI)**

Then reference it from an app setting:

- `PG_CONNSTRING = @Microsoft.KeyVault(SecretUri=<secret-uri>)`

3) “From shell script: Does runtime matter? We plan to use Python for Functions.”

Yes:

- **Logic App Standard** runs on the **Logic Apps runtime** hosted on App Service / Functions runtime.
- Your Function App runtime (Python) is **separate** from Logic App Standard runtime.
- In scripts like `az webapp create --runtime "dotnet" --deployment-container-image-name ...` :
 - That runtime refers to the **hosting web app container/runtime** for that specific app.
 - For a **Python Function App**, create a Function App with Python runtime (not dotnet).

Day 3 — ForEach batch, async response, inserting rows, local debugging

1) “ForEach with batch processing (500 records) error raised”

Common causes:

- Connector throttling / API limits
- Too much parallelism
- Timeouts / large payloads

Fixes:

- Lower ForEach **concurrency**
- Use **pagination** / batching (chunk size)
- Prefer **bulk insert** patterns (stored procedure / COPY / multi-row insert)

- Add retries with exponential backoff where safe

2) “Receives a CSV file via HTTP — how to handle this?”

Two common approaches:

A) multipart/form-data

- HTTP trigger accepts file(s)
- Parse parts, loop through each file

B) JSON with base64

- Client sends { "fileName": "...", "contentBase64": "..." }
- Decode in Function / Logic App (Functions preferred for CSV parsing)

For non-trivial CSV (headers optional, empty lines, quoting), a **Function** is usually better.

3) “When/How to use Asynchronous response?”

Use async response when processing may exceed client timeouts:

- Immediately respond **202 Accepted Consider running**
- Continue work in background (Queue/Service Bus/Blob trigger)
- Provide status endpoint or callback/webhook

In Logic Apps:

- Return a **Response** early (202 + correlation id)
- Use a queue to continue processing
- Or call an async function that enqueues work

4) “Why hard-coded value instead of input from request?”

Best practice:

- Use trigger data, e.g. `@triggerBody()?['x']` or `@triggerOutputs()?['queries']['x']`
- Validate and default missing values
- Keep hard-coded values only for constants (e.g., table name) or config

5) “How to handle Id? Remove id part from code view.”

If DB generates IDs:

- Don't pass `id` in insert; let DB assign it (serial/identity/uuid default) If caller supplies IDs:
- Validate uniqueness and format

6) “Database has created_at column but not included in flow — why?”

Often `created_at` is:

- `DEFAULT now()` in DB
- computed automatically

So it doesn't need to be included in insert, unless you want the client-controlled timestamp.

7) “Insert action — can I run from VS Code?”

Yes for **Logic App Standard**:

- Run workflow locally (VS Code)
- Ensure you have:
 - `connections.json` set up
 - local settings / managed identity alternative (local uses connections)

8) “How to download Logic Apps from Azure to local with dependencies for local debugging?”

For **Logic App Standard**:

- It's a project structure (like Functions). Options:
 - Keep source in repo and deploy from there (recommended)
 - Use Kudu/zip download from the App Service content (`site/wwwroot`) and reconstruct project
- The most reliable approach: maintain the Standard project in source control.

Day 3–4 — Retry policy, URL hardcoding, SAS security, managed identity

1) “Common retry policy for multiple Azure Functions without hardcoding”

Logic Apps doesn't have a single global retry policy, but you can:

- Standardize a **Scope** pattern and reuse workflow templates
- Use **ARM/Bicep** parameters to inject retry policy consistently
- Create a wrapper workflow/function that encapsulates retry logic

Typical retry settings:

- Exponential
- Max attempts (e.g., 3–5)
- Only for transient errors (429/5xx)

2) “Remove hard coded URL. Security concern by SAS code.”

Best practice:

- Use **Managed Identity** for Storage / Key Vault access
- Avoid embedding SAS tokens in code/workflows
- If SAS is required:
 - use short expiry
 - store it in Key Vault
 - rotate regularly

3) “Managed identification” (Managed Identity)

Use managed identity wherever possible:

- Storage: RBAC roles like **Storage Blob Data Contributor**
- Key Vault: Secret Get/List permissions (via RBAC or access policy)
- PostgreSQL: depends on Postgres offering; for Flexible Server, consider Entra ID auth (where supported) or keep creds in KV.

Day 4 — Compose not working, invalid Blob URL, chunking/streaming, blob policies

1) “Compose action not working”

Checklist:

- Expression starts with `@` when using expression mode
- Correct action name: `body('ActionName')`
- Correct path exists (avoid null reference; use `?` safe navigation where supported)

2) “Blob URL was composed as invalid”

Common mistakes:

- Missing `/` between account/container/blob
- Double-encoding blob names
- Passing full URL where connector expects name only

Fix:

- Use `encodeURIComponent()` only on the blob name when building URLs
- Prefer **Blob connector** actions rather than manual URL concatenation

3) “Merge chunk data not working with multiple data blocks / streaming”

Logic Apps are not ideal for true streaming. Better patterns:

- Upload chunks to Blob and merge using:
 - Function App that appends/concat
 - or block blobs (stage blocks + commit block list)
- If you must in Logic Apps:
 - Use Append to string/array variables carefully (watch size limits)
 - Keep payload small; store intermediate state in Blob

4) “Blob versioning and soft delete policies + archive after 10 days”

Use **Storage Account** features:

- Enable **Blob versioning**
- Enable **Soft delete**
- Configure **Lifecycle management** rules:
 - Move to Cool/Archive after X days
 - Delete versions/snapshots after Y days

Blob triggers, trigger conditions, and “if” questions

1) “Why I didn't see any condition for if?”

There are two ways filtering happens:

- Explicit **Condition** action in the workflow
- **Trigger conditions** (in workflow trigger settings) that prevent runs from starting

2) “What’s available for trigger condition on Blob action?”

Trigger condition is an expression that must evaluate true for the run to start. Examples:

- Only CSV files:
 - `@endsWith(triggerBody()?['Path'], '.csv')`
- Only non-empty blobs:
 - `@greater(int(triggerBody()?['Size']), 0)`

3) “How to decide what keyword is required? What else keyword is available or why not in header?”

Depends on the connector/action:

- For HTTP: headers are optional unless the API requires them
- For connectors: required fields are shown in the designer; optional fields appear under “Add new parameter” Use run history and connector documentation to see what is mandatory and what is optional.

4) “Which way is better — pass full path of blob or blob meta?”

Best practice:

- Pass **container + blob name** (or an immutable identifier)
- Fetch metadata when needed (size, content-type, tags) Avoid passing full SAS URLs unless you must.

App settings, tracking, logging/telemetry, dashboards, exceptions, KQL

1) “How to get Appsetting values?”

- Logic App Standard: `@appsetting('SETTING_NAME')`
- Function: `os.getenv("SETTING_NAME")`

2) “Tracking section — what value recommended?”

Use tracked properties for:

- correlation id / request id
- business identifiers (customer_id, document_id) Avoid PII/secrets.

3) “Logging vs telemetry — both to Application Insights?”

- **Application Insights**: traces, requests, dependencies, exceptions
- **Diagnostic settings** can also send to **Log Analytics** Often you use both:
- App Insights for APM
- Log Analytics for centralized operational logs/KQL across services

4) “Create a dashboard with event raised for specific keyword”

Approach:

- Log a structured message (include `eventName`, `correlationId`, etc.)
- Query in Logs (KQL)
- Build an **Azure Monitor Workbook** (or Dashboard) from the query
- Optionally create an **Alert rule** when count > threshold

5) “How to raise custom exception manually for catch block?”

Logic Apps:

- Use **Terminate** action with **Status = Failed** and message
- Wrap steps in a **Scope**; configure `runAfter` on a failure-handling scope

Functions:

- Raise an exception in code; ensure it's logged and returns proper HTTP status.

6) “Hands-on: Create a simple KQL query to find recent errors”

Examples (Log Analytics / App Insights tables vary):

App Insights (typical):

```
exceptions
| where timestamp > ago(1h)
| order by timestamp desc
```

Function/Logic traces:

```
traces
| where timestamp > ago(1h)
| where severityLevel >= 3
| order by timestamp desc
```

Day 5–7 — Mapping/transform, CSV/X12, integration account alternatives

1) “Mapping/transform without Integration Account — Liquid template syntax?”

Options:

- **Azure Function** performs transformation (Liquid/Jinja2/custom code)
- **Logic Apps Standard** can do transformations if you use the right built-in actions/features available in your environment.
- Store mapping templates/config in **Blob Storage** and load at runtime.

Liquid ideas:

- loops for repeated segments
- conditional mapping based on indicators
- output JSON/XML based on template

2) “Parse JSON: why do we need this vs schema on account?”

Parse JSON helps:

- Validate structure early
- Provide typed tokens to the designer (easier expressions)
- Catch missing fields before deeper steps If you already have schema-driven parsing elsewhere, Parse JSON can be optional—but it improves maintainability.

3) “How to handle dynamically mapping and transforming?”

Recommended design:

- Store mapping configs/templates in Blob (or repo)
- Choose mapping based on:
 - client + transactionSet , or
 - a mappingPath parameter, or
 - metadata/tags on the incoming file
- Load mapping template, run transform, store output + status

4) “Sample to use external file (mapping JSON) from Storage account”

Pattern:

1. Blob trigger (incoming file)
2. Determine mapping path
3. Get mapping file from Blob
4. Transform (Function recommended)
5. Write transformed output + status

5) “How to handle multiple files in request?”

- Accept multipart/form-data with multiple file parts
- Loop over files; for each file:
 - persist raw
 - process
 - write status records

6) “CSV input demo if not same as segments”

CSV is row/column oriented:

- Parse using Function (robust CSV handling)
- Map each row into target entity

- Batch inserts for performance

7) “Instruction to host function app locally to debug”

- Install Azure Functions Core Tools
- Run:
 - `func start` / `func host start`
- Use VS Code “Attach to Python Functions” debug config